Deep Reinforcement Learning

Temporal Difference Learning & Q Learning

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

- Recall GPI
- GLIE
- SARSA (on-policy)
- Q-learning (off-policy)

Greedy in the limit with infinite exploration (GLIE)

A learning policy π is called GLIE if it satisfies the following two properties:

- If a state is visited inifinitely often, then each action is choosen inifintely often:

$$\lim_{i \to \infty} \pi_i(a|s) = 1 \quad \forall \{s \in \mathcal{S}, a \in \mathcal{A}\}$$

- In the limit ($i \to \infty$) the learning policy is greedy with respect to the learned action value:

$$\lim_{i \to \infty} \pi_i(a|s) = \pi(s) = \arg\max_{a \in \mathcal{A}} Q(s, a) \quad \forall s \in \mathcal{S}$$

GLIE Monte Carlo control

MC-based contro using ϵ-greedy exploration is GLIE, if ϵ is decreased at a rate

$$\epsilon_i = \frac{1}{i}$$

with i being the increasing episode index. In this case,

$$\hat{Q}(s, a) = Q^*(s, a)$$

follows.

SARSA

- **S**tate-**A**ction-**R**eward-Next-**S**tate-Next-**A**ction
- on-policy

Algorithm: SARSA.
initialize

- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $Q(\text{terminal-state}, \cdot) = 0$
- $\pi = \epsilon\text{-greedy}(Q)$

for $j = 1, 2, \ldots, J$ episodes:
Initialize $s_t \leftarrow s_0, t \leftarrow 0$
while s_t is not terminal:

Take action $a_t \sim \pi(s_t)$ and observe (r_t, s_{t+1})
Select $a_{t+1} \sim \pi(s_{t+1})$
Update Q given $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

Update policy $\pi = \epsilon$-greedy(Q)
$t \leftarrow t + 1$

Convergence of SARSA

[Based on Marius Lindauer's lecture]

SARSA for finite-state and finite-action MDPs converges to the optimal action-value, $Q(s, a) \to Q^*(s, a)$, under the following conditions:

1. The policy sequence $\pi_t(a \mid s)$ satisfies the condition of GLIE
2. The step-sizes α_t satisfy the Robbins-Munro sequence such that

$$\sum_{t=1}^{\infty} \alpha_t = \infty$$

$$\sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

For example, $\alpha_t = \frac{1}{t}$ satisfies the above condition.

Q-Learning

- SARSA estimates Q of the current policy
- Q-learning is similar but directly estimates Q^*
- off-policy update, since the optimal action-value function is updated independent of a given behavior policy

Algorithm: Q-Learning.
initialize

- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $Q(\text{terminal-state}, \cdot) = 0$
- $\pi = \epsilon\text{-greedy}(Q)$

for $j = 1, 2, \ldots, J$ episodes:

Initialize $s_t \leftarrow s_0, t \leftarrow 0$

while s_t is not terminal:

Take action $a_t \sim \pi(s_t)$ and observe (r_t, s_{t+1})

Select $a_{t+1} \sim \pi(s_{t+1})$

Update Q given $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Update policy $\pi = \epsilon$-greedy(Q)

$t \leftarrow t + 1$

Function Approximation

Motivation

- previously we were assuming that we can model everything by table loop-ups

Problems:

- real-world states can be continuous
- often we cannot see all possible states

Solution: Approximate value function, e.g. by

- linear combination of features
- decision trees
- neural networks -> DeepRL !

Value Function Approximation

- Represent state/state-action value function with a parametrized function

Linear Value Function Approximation

Weighted linear combination of features:

$$\hat{V}^{\pi}(s;\boldsymbol{\theta}) = \boldsymbol{\phi}(s)^{\top}\boldsymbol{\theta}$$

Optimization of objective (MSE):

$$J(\boldsymbol{\theta}) = \mathbb{E}\left[\left(V^{\pi}(s) - \hat{V}^{\pi}(s;\boldsymbol{\theta})\right)^2\right]$$

Gradient descent:

$$\Delta(\boldsymbol{\theta}) = -\frac{1}{2}\alpha\nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta})$$

Update rule:

$$\Delta \boldsymbol{\theta} = -\alpha \left(V^{\pi}(s) - \boldsymbol{\phi}(s)^{\top} \boldsymbol{\theta} \right) \boldsymbol{\phi}(s)$$

Monte Carlo Value Function Approximation

Monte Carlo VFA

- G_t is a noisy but unbiased estimate of the true expected return $V^\pi(s_t)$

Update rule:

$$\Delta\boldsymbol{\theta} = -\alpha\left(G_t - \boldsymbol{\phi}(s)^\top\boldsymbol{\theta}\right)\boldsymbol{\phi}(s)$$

Monte Carlo VFA: Convergence

[Based on Marius Lindauer's lecture]

Based on Tsitsiklis and Van Roy. 1997.

Define the mean squared error of a linear value function approximation for a particular policy π relative to the true value as

$$\text{MSVE}(\boldsymbol{\theta}) = \sum_{s \in S} d(s)(V^{\pi}(s) - \hat{V}^{\pi}(s; \boldsymbol{\theta}))^2$$

where

- $d(s)$: stationary distribution of π in the true decision process
- $\hat{V}^{\pi}(s; \boldsymbol{\theta}) = \boldsymbol{\phi}(s)^T \boldsymbol{\theta}$, a linear value function approximation

Monte Carlo policy evaluation with VFA converges to the weights $\boldsymbol{\theta}_{MC}$ which has the minimum mean squared error possible:

$$\text{MSVE}(\boldsymbol{\theta}_{MC}) = \min_{\boldsymbol{\theta}} \sum_{s \in S} d(s)(V^{\pi}(s) - \hat{V}^{\pi}(s; \boldsymbol{\theta}))^2$$

Temporal Difference Learning with Value Function Approximation

TD-Learning with VFA

Compute target using bootstrapping:

$$r_t + \gamma \hat{V}^{\pi}(s_{t+1}; \boldsymbol{\theta})$$

Since target is not updated, $\hat{V}^{\pi}(s_{t+1}; \boldsymbol{\theta})$ is treated as a constant in the derivative.

Update rule:

$$\Delta \boldsymbol{\theta} = -\alpha \left(r_t + \gamma \hat{V}^{\pi}(s_{t+1}; \boldsymbol{\theta}) - \boldsymbol{\phi}(s)^{\top} \boldsymbol{\theta} \right) \boldsymbol{\phi}(s)$$

TD-Learning VFA: Convergence

[Based on Marius Lindauer's lecture]

TD(0) policy evaluation with VFA converges to weights $\boldsymbol{\theta}_{TD}$ which is a constant factor of the minimum mean squared error possible:

$$\text{MSVE}(\boldsymbol{\theta}_{TD}) \leq \frac{1}{1-\gamma} \min_{\boldsymbol{\theta}} \sum_{s \in S} d(s)(V^{\pi}(s) - \hat{V}(s; \boldsymbol{\theta}))^2$$

SARSA and Q-Learning with VFA

[Based on Marius Lindauer's lecture]

Similar to V(s), we can approximate Q(s,a):

$$\hat{Q}(s, a; \boldsymbol{\theta}) = \boldsymbol{\phi}(s, a)^\top \boldsymbol{\theta}$$

Monte Carlo Update:

$$\Delta \boldsymbol{\theta} = -\alpha \left(G_t - \hat{Q}(s, a; \boldsymbol{\theta}) \right) \nabla_{\boldsymbol{\theta}} \hat{Q}(s, a; \boldsymbol{\theta})$$

SARSA with TD target:

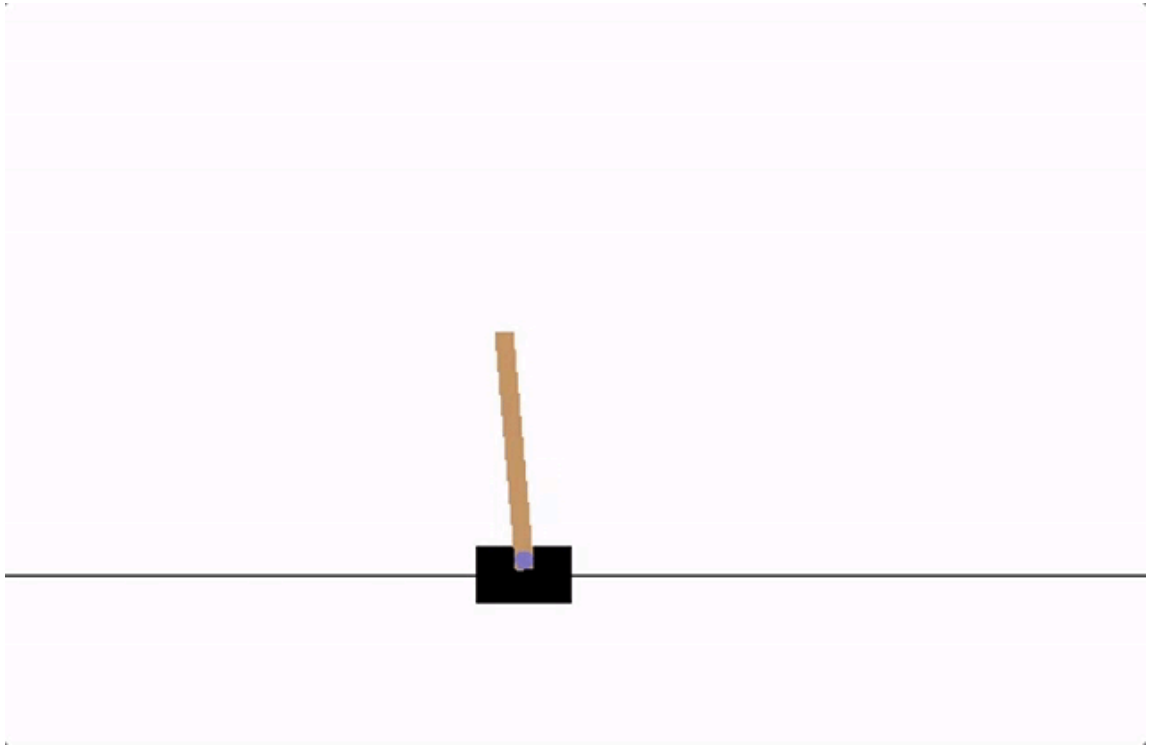
$$\Delta \boldsymbol{\theta} = -\alpha \left(r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1}; \boldsymbol{\theta}) - \hat{Q}(s, a; \boldsymbol{\theta}) \right) \nabla_{\boldsymbol{\theta}} \hat{Q}(s, a; \boldsymbol{\theta})$$

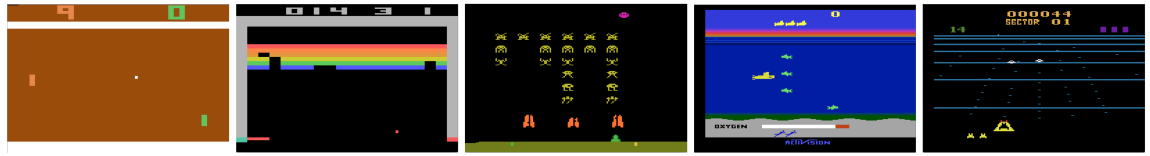
Q-Learning with TD target:

$$\Delta\boldsymbol{\theta} = -\alpha \left(r_t + \gamma \max_a \hat{Q}(s_{t+1}, a; \boldsymbol{\theta}) - \hat{Q}(s, a; \boldsymbol{\theta}) \right) \nabla_{\boldsymbol{\theta}} \hat{Q}(s, a; \boldsymbol{\theta})$$

Deep Q-Learning

Towards more complex learning tasks

CartPole: 

Atari: 

Credit: Mnih et al. 2013

-> Deep Neural Networks (DNNs) as value function approximators

Recall: Online Q-Learning with VFA

1. Take action a_t and observe (s_t, a_t, r_t, s_{t_1}) -> correlated! breaks i.i.d assumption of NNs
2. Update Q-Network:

$$\Delta\boldsymbol{\theta} = -\alpha \left(r_t + \gamma \max_a \hat{Q}(s_{t+1}, a; \boldsymbol{\theta}) - \underbrace{\hat{Q}(s,a;\boldsymbol{\theta})}_{\text{non-stationary!}} \right) \nabla_{\boldsymbol{\theta}} \hat{Q}(s, a; \boldsymbol{\theta})$$

Deep Q-Network (DQN)

1. Correlation -> Replay Buffer from which we sample batches i.i.d.
2. Target network: Copy of previous network weights used as target and updated delayed: $\boldsymbol{\theta}^{-}$

Maximization Bias

- maximum in both value estimation and action selection
- leads to positive bias in Q-values

Double DQN

- keep two q networks and toss coin which one to update

Prioritized Experience Replay

Based on Marius Lindauer's Lecture

- Let i be the index of the i-th tuple of experience (s_i, a_i, r_i, s_{i+1})
- Sample tuples for the update using priority function
- Priority of a tuple i is proportional to DQN error

$$p_i = |r_i + \gamma \max_{a' \in \mathcal{A}} Q(s_{i+1}, a'; \boldsymbol{\theta}^-) - Q(s_i, a_i; \boldsymbol{\theta})|$$

- Update p_i every update. p_i for new tuples is set to the maximum value

- One method: proportional (stochastic prioritization)

$$P(i) = \frac{p_i^\beta}{\sum_k p_k^\beta}$$

- $\beta = 0$ yields random selections

https://arxiv.org/pdf/1511.05952

References